

30th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2026)

Agentic AI for Public Administration: A Multi-Agent Architecture and Controlled Fault Injection Evaluation

Radu-Mihai Gheorghe^{a,b}, Petru-Liviu Bouruc^b, Ciprian Paduraru^b, Alin Stefanescu^{b,c}

^aUiPath, Romania

^bUniversity of Bucharest, Romania

^cInstitute for Logic and Data Science, Romania

Abstract

The digital transformation of public administration requires intelligent systems that guide citizens through complex regulatory workflows while remaining reliable under the integration faults that arise in production environments. Agentic AI systems, in which large language models coordinate specialised agents across multi-step workflows, offer a promising path toward unified interfaces for fragmented public services, yet their robustness under realistic service failures remains poorly understood.

This paper presents a multi-agent architecture for public administration services and evaluates its robustness under controlled fault injection conditions, with the goal of quantifying how many integration failures propagate silently to citizens and assessing the financial feasibility of institutional adoption.

The architecture is instantiated for Romanian tax and fiscal administration using the open-source UiPath Python SDK and LangGraph. It includes an intent-routing orchestration layer, five domain-specific agents covering freelancer tax contributions under the PFA (*persoană fizică autorizată*, in Romanian; self-employed status) regime, property sale taxation, rental income obligations, fiscal certificate issuance, and electronic invoicing through RO e-Factura, together with a shared services layer for retrieval-augmented generation over the Romanian Tax Code, deterministic tax computation, document processing, and payment handling. Robustness is evaluated through a controlled fault injection framework that instruments 19 external service boundary operations with four chaos operators: timeout, delay, partial response, and corruption, applied at configurable rates with a seeded random number generator for reproducibility.

Experiments over 100 individual boundary calls reveal a structural silent pass-through rate of approximately 63%: faults that do not affect the downstream branching predicate remain undetected, with the LLM producing plausible citizen-facing responses despite null or corrupted service data. This rate is determined by the operator mix and response schema structure, not by the injection rate, and remains stable across all tested values of the injection probability. LLM inference costs are estimated at approximately \$0.05 per session, scaling to about \$5,060 per month for a large agency handling 5,000 daily interactions. The 63% silent pass-through rate motivates output-level contract verification as a necessary complement to agentic systems deployed in regulated public services. The implementation and evaluation artefacts are released publicly to support reproducibility and further research.

© 2026 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the KES International.

Keywords: multi-agent systems ; agentic AI ; robustness of agentic AI systems ; fault injection ; chaos engineering ; public administration ; tax automation ; UiPath SDK ; LangGraph

1. Introduction

The digital transformation of public administration has become a strategic priority across Europe. In Romania, recent initiatives include Romania's National Agency for Fiscal Administration (ANAF) *Spațiul Privat Virtual* (SPV; Virtual Private Space, in Romanian) portal for tax declarations, Ghiseul.ro, the national electronic payment platform, and RO e-Factura, the national electronic invoicing system. Although these platforms support important administrative functions, they also impose a substantial burden on citizens and businesses. Users must navigate several interfaces, interpret changing tax regulations, and coordinate across disconnected services to complete routine fiscal tasks. Agentic AI systems, in which large language models (LLMs) coordinate specialised agents, interact with external services, and maintain context across multi-step workflows, offer a plausible way to provide unified conversational access to such fragmented processes [1, 2].

The architecture addresses two connected stakeholder needs. For Romanian taxpayers, including individuals and small businesses, the goal is to provide a single conversational interface for routine fiscal tasks that are currently distributed across SPV, Ghiseul.ro, and RO e-Factura. The interface is therefore intended to reduce procedural fragmentation rather than replace the underlying public platforms. For ANAF, as the institutional provider of such a service, the goal is to maintain coherent behaviour when dependent services are slow, partially available, or return malformed data. This distinction structures the rest of the paper: the architecture addresses the citizen-facing access problem, while the controlled fault injection study evaluates robustness at service boundaries.

Deploying such a system in practice requires more than an LLM orchestration library. A bare LangChain or LangGraph implementation still leaves several engineering concerns unresolved, including model access, retrieval infrastructure, retry logic for unreliable integrations, human escalation, and document storage with access control. These concerns become especially important in public administration, where workflows often span multiple systems and where some government portals still depend on browser-based interaction rather than public APIs. In such settings, a practical implementation substrate must support both agent orchestration and operational integration.

The present prototype uses the open-source UiPath Python SDK [3] together with LangGraph [4]. This choice is motivated by integration requirements rather than platform preference alone. The SDK supports native LangGraph state machines while also providing orchestration, persistence, monitoring, storage, human-in-the-loop escalation, and retrieval services within the same execution environment. The UiPathChat interface exposes LLM access through the platform gateway, while ContextGroundingRetriever supports retrieval-augmented generation over indexed knowledge bases. The same environment also supports queue-based execution, task creation for human review, and storage buckets for document handling. This reduces the need to assemble and maintain a separate stack for model serving, vector search, workflow state, and operational controls. Within a single execution trace, an agent can classify user intent, retrieve relevant fiscal guidance, compute tax obligations through deterministic services, and coordinate interactions with systems such as SPV and Ghiseul.ro.

However, a workable implementation substrate does not by itself address robustness. In public administration, users expect stable behaviour even when a dependent system is slow, partially unavailable, or returns malformed data. These are ordinary integration failures in production environments, but they are not always exercised explicitly during development. An agent that reacts only to a service-reported success flag may still generate a plausible response when the underlying payload is incomplete or corrupted. This creates a particularly problematic failure mode in regulated settings: the system appears coherent at the interface level while silently propagating invalid state. Analysing this behaviour requires controlled perturbation at the system boundaries, in the spirit of chaos engineering for service-based software [5].

This paper makes three contributions:

1. A multi-agent architecture for public administration is introduced, in which an Entry Agent performs intent detection and routes requests to one of five domain-specific agents: PFA/Freelancer, Property Sale, Rental Income, Certificate, and E-Factura. These agents share services for document intake, retrieval over the Romanian Tax Code, deterministic tax computation, and payment processing through national platforms such as SPV, Ghiseul.ro, and RO e-Factura.
2. A fault injection framework is defined for 19 external service boundary operations using four chaos operators: timeout, delay, partial response, and corruption. The framework supports configurable injection rates and deterministic replay through a seeded random number generator. Across injection rates from 0% to 100%, the

experiments show that approximately 63% of injected faults propagate silently when the affected fields do not alter the downstream branching predicate.

3. An openly available implementation of the architecture and evaluation setup is provided to support reproducibility and further research on robust agentic systems for regulated public services.

The architecture follows a hierarchical supervisor topology [6] implemented with LangGraph's StateGraph and conditional routing. Its orchestration and shared services layers are designed to be reusable across public service domains, while the present instantiation focuses on Romanian tax and fiscal administration: freelancer taxation under the PFA (*persoană fizică autorizată*, in Romanian; self-employed status) regime and *Declarația Unică* (D212; Single Tax Return, in Romanian), property transactions, rental income, fiscal certification, and electronic invoicing. These workflows provide a representative example of the procedural fragmentation and regulatory complexity that public-facing intelligent information systems must address.

The central empirical result is structural rather than incidental: many faults remain invisible to the agent when they do not affect the predicate used for control flow. This motivates output-level validation as a necessary complement to agentic systems deployed in regulated public services.

2. Related Work

Agentic AI for digital government and tax services. The use of NLP and AI in public administration has evolved from document classification and information extraction toward conversational systems that support multi-step administrative tasks. Jiang et al. [7] identify automation, extension, and transformation as the main contribution patterns across 1,032 NLP studies in government contexts. Madan and Ashok [8] review barriers to AI adoption in public-sector organisations and emphasise transparency, accountability, and institutional readiness. In tax administration, Nay et al. [9] show that GPT-4 combined with retrieval over legal materials can achieve competitive performance on tax-law question answering. At the institutional level, Schmitz et al. [10] identify governance requirements that should be satisfied before agentic AI systems are adopted in public-sector practice. The present work extends this line of research from advisory question answering to a multi-agent service architecture that supports several operational tax workflows in a concrete national setting.

Multi-agent architectures and retrieval-augmented generation. Wang et al. [11] describe LLM-based autonomous agents through a common decomposition into profile, memory, planning, and action modules. AutoGen [6] and LangGraph [4] represent two influential implementation styles: the former emphasises conversational multi-agent interaction, whereas the latter provides a typed state-machine model suited to explicit workflow control and long-running execution. The architecture presented here adopts LangGraph's hierarchical supervisor pattern with intent-gated routing. For knowledge grounding, Gao et al. [12] provide a broad survey of retrieval-augmented generation, while Pipitone and Houir Alami [13] examine retrieval quality in legal-domain RAG. Their findings are directly relevant to tax and regulatory assistance, where retrieval precision strongly affects answer quality and traceability.

Robustness, fault injection, and reliability of LLM agents. AgentBench [14] evaluates LLMs as agents across several environments and highlights long-horizon reasoning and instruction following as recurring weaknesses. Rabanser et al. [15] propose a set of reliability metrics spanning consistency, robustness, and predictability, and show that stronger capability does not automatically yield stronger reliability. Shah et al. [16] study faults in open-source agentic repositories and identify failures at the boundary between probabilistic model outputs and deterministic program logic as a dominant class. This is the same interface targeted in the present evaluation. Owotogbe [17] applies chaos engineering ideas to LLM-based multi-agent systems through hallucination and communication faults, and Gupta [18] extends this direction with ReliabilityBench, which includes timeout, partial-response, and schema-drift conditions. The contribution here differs in three respects: it studies a regulated public-service setting, it uses a decorator-based injection mechanism that remains separate from agent logic, and it connects the observed silent failure rate to the structure of the response schema and branching predicate.

At the European policy and implementation level, Tangi et al. [19] discuss future directions for advancing AI adoption in EU public administrations, with emphasis on organisational capacity, regulatory alignment, and high-impact public-sector application domains.

Research gap. Existing work leaves three connected gaps. First, studies on AI for government and tax services have mainly addressed advisory question answering, document processing, or institutional adoption. They do not provide an operational multi-agent architecture that combines intent routing, retrieval over tax-code content, deterministic computation, document handling, and payment support in a concrete national fiscal setting. Second, work on multi-agent orchestration and retrieval-augmented generation provides useful implementation patterns, but it rarely examines how these systems behave when external services fail. Third, robustness studies and fault injection benchmarks for LLM agents usually target generic or synthetic settings. They do not quantify how many boundary faults can pass silently through a regulated public-service workflow, nor do they relate this behaviour to response schemas and branching predicates. The present paper addresses this intersection by combining a concrete public-administration architecture with a controlled boundary-level fault injection evaluation.

3. Methods

The study follows a two-phase research method. The first phase is design and construction: a multi-agent architecture is developed to address the fragmentation of Romanian fiscal services and to provide a unified conversational interface grounded in tax-code content. The second phase is controlled evaluation: integration faults are injected at external service boundaries under configurable rates, and the resulting agent behaviour is classified as detectable failure or silent pass-through. This design separates the architectural contribution from the robustness evaluation and makes the unit of analysis explicit.

This section describes the multi-agent architecture for public administration, instantiated here for Romanian tax and fiscal services, together with the fault injection framework used to evaluate robustness under simulated service failures. The system is organised into five layers: user interface, orchestration, domain agents, shared services, and infrastructure. A separate fault injection layer instruments mocked external boundaries so that controlled perturbations can be introduced during development and testing without requiring live access to government portals.

3.1. System architecture

The architecture follows a hierarchical supervisor pattern implemented as a LangGraph StateGraph with typed state and conditional routing. Figure 1 shows the full organisation of the system across the five layers.

The system maintains a shared state across graph nodes. This state includes the user query, conversation history, routing fields, intent classification results, workflow status, and a context dictionary for cross-agent data exchange. The graph takes a user query as input and returns the response text together with the detected intent, confidence score, and workflow status.

3.1.1. Orchestration layer: Entry Agent

The Entry Agent is the sole entry point for user interaction and is responsible for intent detection. Given a user query, it invokes an LLM (GPT-4o via the UiPath LLM Gateway) with a system prompt that lists the supported service categories and requests a structured classification. The classifier distinguishes among the supported tax-service workflows, general questions, and unclear requests. It returns the predicted intent, a confidence score, explanatory reasoning, and any domain-specific entities extracted from the user query, such as annual income, property value, ownership duration, monthly rent, or invoice type.

A deterministic mapping translates the classified intent into the corresponding graph node. Routing is then gated by a confidence threshold. If the confidence score falls below the configured threshold, the request is redirected to a clarification node even if an intent has been assigned. This two-stage procedure combines structured LLM classification with deterministic routing and reduces the risk of dispatching ambiguous requests to the wrong domain agent.

3.1.2. Domain agents layer

Five domain-specific agents implement the main tax-service workflows. Each agent receives the shared state, invokes the LLM with a domain-specific prompt, interacts with shared services when necessary, and returns updated state together with response messages.

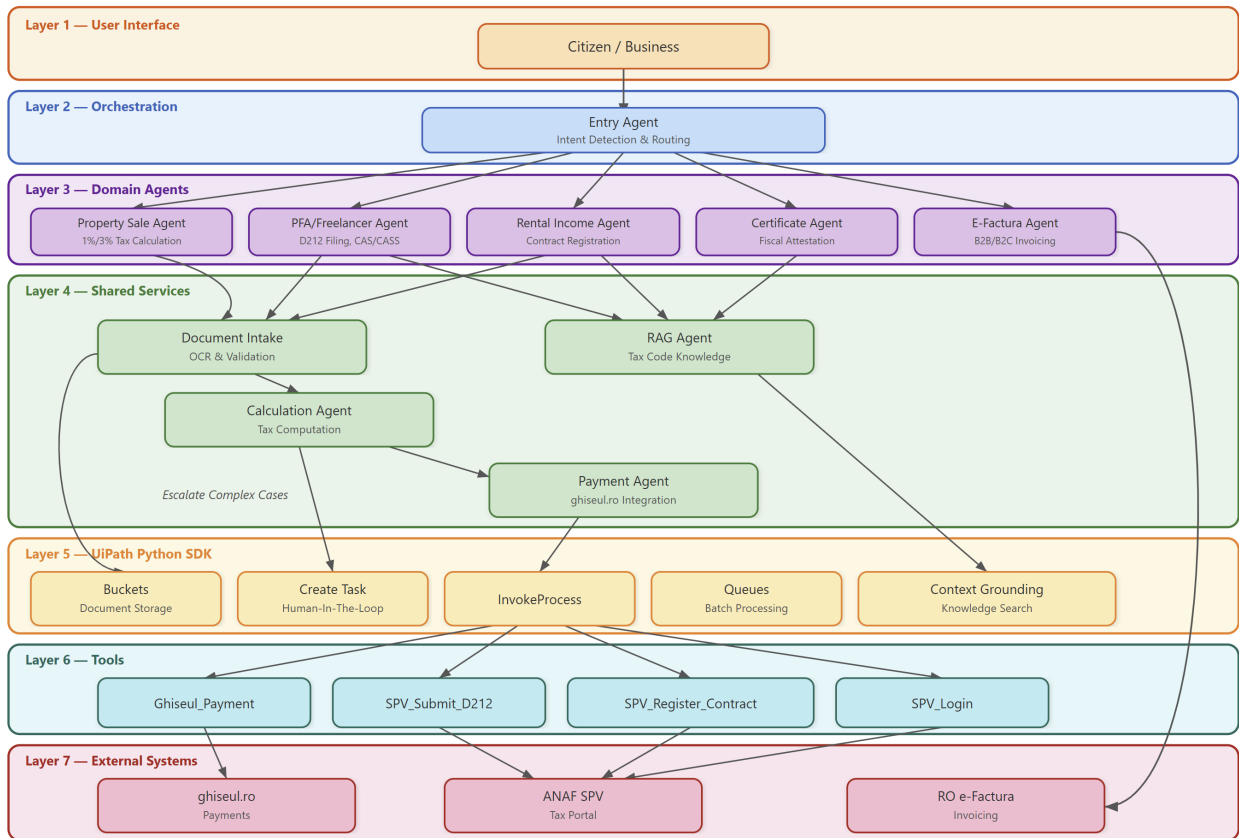


Fig. 1. Multi-agent architecture for Romanian tax administration. The orchestration layer routes citizen requests through an Entry Agent to domain-specific agents. These agents use shared services for document processing, knowledge retrieval, tax computation, and payments. The infrastructure layer exposes UiPath Orchestrator capabilities and external government systems.

PFA/Freelancer Agent. This agent supports filing of *Declarația Unică* (D212; Single Tax Return, in Romanian) for self-employed individuals under the PFA regime. It collects income information from user input or extracted entities, delegates social insurance contribution (CAS, pension, 25%) and health insurance contribution (CASS, 10%) computation to the Calculation Service, and guides the user through submission via SPV. CAS applies when annual income is at least 12× the minimum gross salary (48,600 RON for 2026) and is computed on the threshold amount. CASS applies when annual income is at least 6× the minimum gross salary (24,300 RON for 2026).

Property Sale Agent. This agent computes property sale tax according to ownership duration: 3% for properties held for less than 3 years and 1% for properties held for 3 years or more. It extracts the property value and holding period, delegates the calculation, and provides payment guidance through Ghiseul.ro.

Rental Income Agent. This agent supports rental contract registration with ANAF and computes the 10% flat tax on annual rental income. The workflow includes contract data collection, registration through SPV, and payment support.

Certificate Agent. This agent assists users in obtaining a *Certificat de atestare fiscală* (fiscal attestation certificate, in Romanian). It identifies the certificate type, checks document requirements, and guides the submission process.

E-Factura Agent. This agent supports electronic invoicing for both B2B and B2C scenarios. It handles invoice data collection, XML generation consistent with the RO e-Factura standard, submission to the national system, and status monitoring.

All domain agents produce a standardised response field from the final model output, which ensures consistent formatting across workflows.

3.1.3. Shared services layer

Four shared services provide reusable capabilities across all domain agents.

Document Intake Service. This service processes uploaded documents through OCR and validates extracted data against document-specific schemas. It supports D212 forms, rental contracts, and invoices, and returns structured data together with confidence scores and validation errors. Documents with OCR confidence below 80% are flagged for manual review.

RAG Service. This service implements retrieval-augmented generation over dedicated indices containing relevant sections of the Romanian Tax Code and procedural guidance. For a given query, it retrieves the most relevant passages, builds a grounded prompt, and generates an answer conditioned on the retrieved evidence. This supports traceable tax-code interpretation instead of relying solely on parametric model knowledge.

Calculation Service. This service performs deterministic tax computations using decimal arithmetic to avoid floating-point error in financial calculations. It supports PFA contributions, property sale tax, and rental income tax. All calculations depend on configurable parameters such as minimum salary levels and tax rates, allowing annual regulatory updates without changing code.

Payment Service. This service integrates with Ghiseul.ro for tax payment processing. It constructs payment requests from the shared context, initiates the payment, and returns transaction details such as a redirect URL for payment completion.

3.1.4. Infrastructure layer: UiPath platform integration

The infrastructure layer connects the agent system to the UiPath Orchestrator platform and, through mocked boundary adapters, to the target government systems. It provides support for robotic process execution, human review tasks, queue-based batch handling, persistent document storage, and retrieval over indexed knowledge sources.

The tools layer exposes mocked functions that mirror the request and response schemas of the corresponding government services, including SPV authentication, D212 submission, rental contract registration, and payment initiation. In production, these functions would be replaced by UiPath robotic processes. The present prototype uses mock implementations to support controlled evaluation while preserving the external interaction structure of the target workflows.

3.2. Fault injection layer

Production assurance for agentic systems requires robustness not only under nominal operation but also under the integration faults common in deployed environments, including timeouts, partial responses, payload corruption, and communication delays [5]. To study these conditions without relying on live government portals, the mocked external boundaries are instrumented with a controlled fault injection framework that applies chaos operators at configurable rates with deterministic reproducibility.

The fault injector is parameterised by a random seed s , a per-call injection probability $\eta \in [0, 1]$, and a maximum induced latency $\Delta t_{\max}$. Each boundary crossing is treated as an independent trial: at most one fault is injected per call. Specifically, the injector draws a single Bernoulli sample with parameter η ; if the draw fails (probability $1 - \eta$) the payload is returned unchanged, and if it succeeds, one of four operators is selected uniformly at random and applied to that call alone. Within a multi-step citizen session that crosses k boundaries, up to k faults may accumulate across calls, but they are independent and do not overlap. If injection occurs, the selected operator is one of:

- **Timeout:** raises a `TimeoutError` and returns no payload;
- **Delay:** sleeps for a duration sampled uniformly from $[1, \Delta t_{\max}]$ ms and then returns the original payload;
- **Partial response:** selects one payload field uniformly at random and sets it to `None`; and
- **Corruption:** selects one field and replaces its value with a fixed sentinel string.

Fault injection is implemented as a separate wrapper layer around mocked service boundaries, so that the nominal service logic remains unchanged and injection can be disabled by setting $\eta = 0$. Each injected event is logged together with the affected operation and operator type, which enables deterministic replay for a fixed seed.

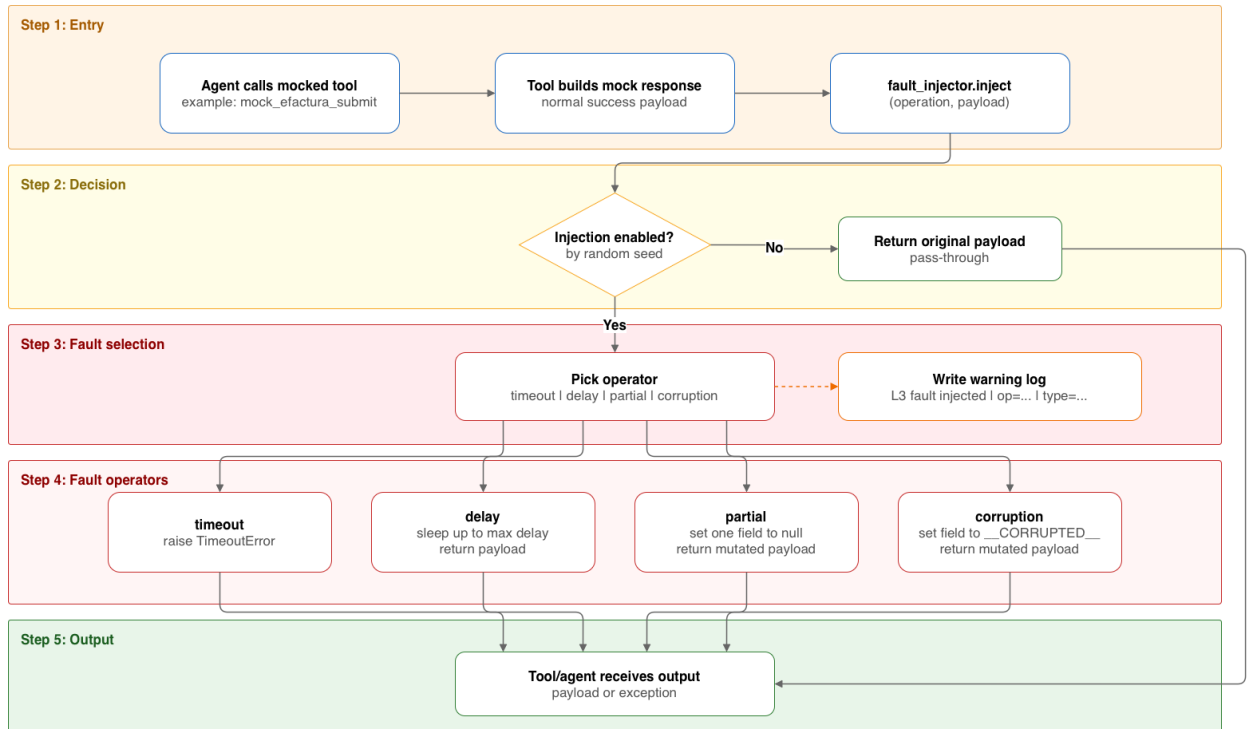


Fig. 2. Fault injection decision flow. When an agent calls a mocked tool, the tool first constructs its nominal response and passes it to the fault injector. A seeded Bernoulli trial determines whether injection occurs. On pass-through, the original payload is returned unchanged. Otherwise, one of four chaos operators is selected uniformly: *timeout* (raises `TimeoutError`), *delay* (sleeps and returns the payload), *partial* (sets one field to null), or *corruption* (replaces one field with a sentinel value). A warning is written to the fault log before the result is returned to the agent.

4. Evaluation

This section describes the prototype implementation, the experimental configuration, and the criteria used to assess whether injected faults were detected or passed through silently. The evaluation is intended as a controlled robustness study of fault behaviour at service boundaries rather than as a broad performance benchmark over large-scale user traffic.

4.1. Prototype and experimental setup

The prototype is implemented in Python 3.11 with LangGraph 0.2, langchain-openai for LLM calls (GPT-4o through the UiPath LLM Gateway), and pydantic-settings for configuration management. All external government interactions are routed through the mocked boundary layer, so no live network calls are required during evaluation. The implementation includes 8 function-level mock tool wrappers and 11 class-method wrappers across three mocked external-system stubs, corresponding to the service boundaries exercised by the five demonstration workflows. Fault injection is applied through a separate wrapper layer around these mocked boundaries, leaving the nominal service logic unchanged. Each injected event is logged with the affected operation and operator type, which supports deterministic replay for a fixed seed. To characterise the effect of injected faults, the experiments executed $N = 100$ individual boundary-crossing calls, each constituting a single independent trial that receives at most one injected fault, sampled across the 8 function-level mock tools in proportion to their invocation frequency in the five demo scenarios. The unit of analysis is the individual call, not the scenario session; a complete citizen workflow may invoke several boundary operations in sequence, each getting its own independent Bernoulli draw, but Table 1 reports per-call counts throughout. Five injection rates were evaluated, $\eta \in \{0.00, 0.25, 0.50, 0.75, 1.00\}$, using a fixed seed $s = 17$ and $\Delta t_{\max} = 250$ ms. An injected fault was classified as *detectable* if the downstream agent response correctly

signalled failure through exception propagation, an explicit error message, or a degraded-service notice. Otherwise, it was classified as a *silent pass-through*.

A fault is structurally silent in two main cases:

- the **delay** operator preserves the payload and changes only the latency; or
- the **partial** or **corruption** operator affects a non-critical field, so that the downstream success predicate (typically `status == "success"`) still evaluates to true and the LLM continues with a plausible response despite incomplete or corrupted context.

4.2. Qualitative results

One representative trace involves a D212 submission initiated by a citizen reporting PFA income of 150,000 RON. The Entry Agent classified the request as `pfa_d212_filing` with confidence 0.95 and routed it to the PFA Agent. The agent computed CAS = 9,900 RON and CASS = 1,980 RON, for a total of 11,880 RON, and then requested confirmation for submission. On the confirmation turn, the SPV submission call was intercepted with $\eta = 1.0$ and $s = 17$, producing:

```
[WARN] fault injected | op=mock_spv_submit_d212 | type=partial
```

The partial operator selected the message field of the SPVSubmissionResult dataclass uniformly at random and set it to None. Because the status field remained "success", the agent followed the success branch and the LLM generated the Romanian message "*Declarația D212 a fost depusă cu succes! ID: D212-01B66C85C3, Timestamp: 2026-03-27T22:20:32.*", which translates as "The D212 declaration was successfully submitted! ID: D212-01B66C85C3, Timestamp: 2026-03-27T22:20:32.". The user therefore received a plausible success confirmation even though one field of the response object had been nullified. This is a silent pass-through: the injected fault altered the service payload but did not alter control flow or surface in the final answer.

For a single-field perturbation on a response object with $|\mathcal{F}|$ fields, the probability that the perturbation leaves the branching field unchanged is $1 - \frac{1}{|\mathcal{F}|}$. For the four-field SPVSubmissionResult, this value is 0.75. This example therefore reflects a structural property of the response schema rather than an isolated error.

4.3. Quantitative results

Table 1 reports operator counts and pass-through rates across the five injection rates. Columns TO, DL, PT, and CR denote timeout, delay, partial, and corruption faults, respectively.

Table 1. Fault injection results ($N = 100$ calls, $s = 17$, $\Delta t_{\max} = 250$ ms).

η	Faults	TO	DL	PT	CR	Silent	Detectable
0.00	0	0	0	0	0	0 (—)	0 (—)
0.25	25	6	6	7	6	16 (64%)	9 (36%)
0.50	50	12	13	12	13	32 (64%)	18 (36%)
0.75	75	19	19	18	19	47 (63%)	28 (37%)
1.00	100	25	25	25	25	63 (63%)	37 (37%)

Across all non-zero injection rates, approximately 63% of injected faults propagated silently. This matches the theoretical expectation under the operator mix used here. Delay faults account for 25% of injected operators and are always silent because they preserve payload content. Partial and corruption faults each account for 25% and are silent with probability 0.75 when the perturbed field is not the one used for branching. The expected silent rate is therefore $0.25 + 2 \times 0.25 \times 0.75 = 0.625$. Timeout faults form the remaining 25% and are always detectable because they raise an exception. Since this rate is determined by the operator distribution and the response-schema structure, it remains stable across values of η : higher injection rates increase the number of silent faults, but not their proportion.

Delay faults introduced a mean overhead of 125.5 ± 72.2 ms per affected call, consistent with the theoretical mean for uniform sampling over $[1, 250]$ ms. At $\eta = 0.50$, this yields an expected per-scenario latency increase of approximately 65 ms after accounting for mock-call frequency.

The quantitative results show that many realistic integration failures remain invisible when agent behaviour depends only on a service-reported success flag. This motivates a separate response-validation layer that checks outputs against explicit contracts rather than relying solely on the service's own status indicator.

4.4. Cost feasibility

Operational cost is an important consideration for public-sector deployment. To provide an illustrative estimate, the analysis uses the GPT-5 public pricing snapshot available at the time of the experiment: \$1.25 per 1 M input tokens and \$10.00 per 1 M output tokens. The estimate is intended to indicate order of magnitude rather than a fixed deployment cost, since model prices and institutional contracts may change over time. Although the prototype uses GPT-4o, the cost estimate uses GPT-5 as an illustrative deployment scenario based on currently available pricing.

A typical user session includes: (i) one Entry Agent call for intent classification (approximately 2,000 input tokens and 200 output tokens), (ii) two to three domain-agent turns (approximately 5,000 input tokens per turn and 500 output tokens per turn), and (iii) one retrieval-grounded generation step (approximately 3,000 input tokens and 400 output tokens). This yields an estimated total of approximately 20,000 input tokens and 2,100 output tokens per session, corresponding to a cost of about \$0.05 per interaction.

LLM inference costs for these institutions scale at approximately **\$1.01 per daily session per month**, assuming 22 working days. Monthly costs therefore range from about \$51 for a small office handling 50 sessions per day to about \$5,060 for a large agency handling 5,000 sessions per day.

These figures exclude platform licensing, infrastructure, and maintenance, which depend on institutional context. Even so, they suggest that LLM inference alone is unlikely to be the main barrier to deployment. In practice, costs could be reduced further by assigning smaller models to routing and intent-classification tasks while reserving GPT-5 for domain reasoning and retrieval-grounded responses.

5. Conclusion

The paper presented a multi-agent architecture for public administration, instantiated for Romanian tax and fiscal services, together with a fault injection framework for robustness evaluation at external service boundaries. The architecture covers five workflows: freelancer tax contributions, property sale tax, rental income, fiscal certificates, and electronic invoicing. These workflows are coordinated through a LangGraph state machine with intent-based routing, retrieval-grounded domain agents, and shared services for tax computation, document processing, and payment handling. The fault injection framework instruments 19 boundary operations with four chaos operators applied at configurable rates and supports deterministic replay through a seeded random number generator, while remaining separate from the mock implementations.

The evaluation showed a structural silent pass-through rate of approximately 63%. Faults that do not alter the branching predicate used by downstream agent logic may remain invisible to both the system and the user, even when the response object contains null or corrupted fields. This result identifies a concrete assurance gap in agentic public-service systems: interface coherence is not sufficient evidence of execution correctness.

This gap is consistent with the broader argument of Paduraru et al. [20], who propose trace-based assurance through structured message-action traces, explicit contracts, deterministic replay, and action-level governance. In the present setting, such mechanisms would make it possible to detect conditions that are currently silent, for example missing submission identifiers, invalid payment codes, or incomplete certificate responses, even when the service reports success and the agent continues normally.

The cost analysis suggests that LLM inference is not prohibitive under the assumptions used here, with estimated monthly costs of about \$5,060 for 5,000 daily sessions.

A natural next step is to integrate output contracts into the present architecture so that agent responses can be checked against explicit validity conditions before they are shown to citizens. This would allow silent payload faults to be detected even when control flow is unchanged and would provide a stronger basis for assessing whether such systems are suitable for regulated public-service settings.

The implementation and evaluation artefacts are publicly available at <https://github.com/radugheo/agentic-public-administration>.

Acknowledgements: This work was partially supported by the project “Romanian Hub for Artificial Intelligence (HRIA)”, funded under the Smart Growth, Digitization and Financial Instruments Programme 2021–2027, MySMIS no. 351416.

References

- [1] M. Z. Pan, N. Arabzadeh, R. Cogo, Y. Zhu, A. Xiong, L. A. Agrawal, H. Mao, E. Shen, S. Pallerla, L. Patel, S. Liu, T. Shi, X. Liu, J. Q. Davis, E. Lacavalla, A. Basile, S. Yang, P. Castro, D. Kang, J. E. Gonzalez, K. Sen, D. Song, I. Stoica, M. Zaharia, M. Ellis, Measuring agents in production (2025). [arXiv:2512.04123](#), [doi:10.48550/arXiv.2512.04123](#).
- [2] World Economic Forum, Capgemini, [AI Agents in Action: Foundations for Evaluation and Governance](#), Tech. rep., World Economic Forum, white paper (Nov. 2025).
URL <https://www.weforum.org/publications/ai-agents-in-action-foundations-for-evaluation-and-governance/>
- [3] UiPath, [UiPath Python SDK](#), online documentation, accessed 2026-04-17 (2026).
URL <https://uipath.github.io/uipath-python/>
- [4] LangChain, [LangGraph overview](#), online documentation, accessed 2026-04-17 (2026).
URL <https://docs.langchain.com/oss/python/langgraph/overview>
- [5] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41. [doi:10.1109/MS.2016.60](#).
- [6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, C. Wang, AutoGen: Enabling next-gen LLM applications via multi-agent conversation (2023). [arXiv:2308.08155](#), [doi:10.48550/arXiv.2308.08155](#).
- [7] Y. Jiang, P. C. I. Pang, D. Wong, H. Y. Kan, Natural language processing adoption in governments and future research directions: A systematic review, *Applied Sciences* 13 (22) (2023) 12346. [doi:10.3390/app132212346](#).
- [8] R. Madan, M. Ashok, AI adoption and diffusion in public administration: a systematic literature review and future research agenda, *Government Information Quarterly* 40 (1) (2023) 101774. [doi:10.1016/j.giq.2022.101774](#).
- [9] J. J. Nay, D. Karamardian, S. B. Lawsky, W. Tao, M. Bhat, R. Jain, A. T. Lee, J. H. Choi, J. Kasai, Large language models as tax attorneys: a case study in legal capabilities emergence, *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 382 (2270) (2024) 20230159. [doi:10.1098/rsta.2023.0159](#).
- [10] C. Schmitz, J. Rystrom, J. Batzner, Oversight structures for agentic AI in public-sector organizations, in: *Proc. of the 1st Workshop for Research on Agent Language Models (REALM 2025)*, Association for Computational Linguistics, 2025, pp. 298–308. [doi:10.18653/v1/2025.realm-1.21](#).
- [11] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, J.-R. Wen, A survey on large language model based autonomous agents, *Frontiers of Computer Science* 18 (6) (2024) 186345. [doi:10.1007/s11704-024-40231-1](#).
- [12] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey (2024). [arXiv:2312.10997](#), [doi:10.48550/arXiv.2312.10997](#).
- [13] N. Pipitone, G. Houir Alami, LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain (2024). [arXiv:2408.10343](#), [doi:10.48550/arXiv.2408.10343](#).
- [14] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, J. Tang, [AgentBench: Evaluating LLMs as agents](#), in: 12th Int. Conf. on Learning Representations (ICLR'24), 2024, conference version. Preprint DOI: 10.48550/arXiv.2308.03688.
URL <https://openreview.net/forum?id=zAdUB0aCTQ>
- [15] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, A. Narayanan, Towards a science of AI agent reliability (2026). [arXiv:2602.16666](#), [doi:10.48550/arXiv.2602.16666](#).
- [16] M. B. Shah, M. M. Morovati, M. M. Rahman, F. Khomh, Characterizing faults in agentic AI: A taxonomy of types, symptoms, and root causes (2026). [arXiv:2603.06847](#), [doi:10.48550/arXiv.2603.06847](#).
- [17] J. Owotogbe, Assessing and enhancing the robustness of LLM-based multi-agent systems through chaos engineering (2025). [arXiv:2505.03096](#), [doi:10.48550/arXiv.2505.03096](#).
- [18] A. Gupta, ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions (2026). [arXiv:2601.06112](#), [doi:10.48550/arXiv.2601.06112](#).
- [19] L. Tangi, P. Rodriguez Müller, S. Schade, A.-A. André, M. Combetto, M. Daoud, Advancing AI adoption in EU public administrations: Future directions and opportunities under the apply AI strategy, Tech. Rep. JRC143539, Publications Office of the European Union, Luxembourg (2026). [doi:10.2760/3112501](#).
- [20] C. Paduraru, P.-L. Bouruc, A. Stefanescu, A trace-based assurance framework for agentic AI orchestration: Contracts, testing, and governance (2026). [arXiv:2603.18096](#), [doi:10.48550/arXiv.2603.18096](#).